TradeX B.Tech Capstone — Technical Defense Guide

Person 4: Algorithmic Trading, Market Intelligence & Advanced DSA Architect

CANDIDATE / DEFENDER: Person 4 **PRESENTATION TIME:** 5 to 7 Minutes

ENGINEERING ROLE: Algorithmic Trading, Market Intelligence & Advanced DSA Architect

CORE DOMAIN: Stock Catalog & LRU Caching, Prefix Trie Autocomplete, Sector-Equity Graph (BFS/DFS), Sorting Suite (6 Algorithms), Searching Suite (Linear vs Binary), Sliding Window & Two-Pointer Momentum, Greedy Capital Allocation, Dynamic Programming Stock Profit Maximizer, Doubly Linked List & Stack Audit Trails

ASSIGNED REPOSITORY FILES:
- **backend/include/services/StockService.hpp**
- **backend/include/services/CacheService.hpp**
- **backend/include/dsa/Trie.hpp**
- **backend/include/dsa/MarketGraph.hpp**
- **backend/include/dsa/SortingAlgorithms.hpp**
- **backend/include/dsa/SearchingAlgorithms.hpp**
- **backend/include/dsa/MarketAnalyticsService.hpp**
- **backend/include/dsa/GreedyPortfolioService.hpp**
- **backend/include/dsa/DynamicProgrammingService.hpp**
- **backend/include/dsa/LinkedList.hpp**
- **backend/include/dsa/Stack.hpp**
- **backend/tests/test_dsa.cpp**

1. Executive Summary & Architectural Ownership

Person 4 represents the pure algorithmic intelligence of the TradeX platform. This engineer designs and maintains the rich Data Structures and Algorithms (DSA) embedded natively in C++. Person 4 replaces slow database full-table scans with an in-memory Prefix Tree (Trie) for O(L) stock search, models market contagion with an Adjacency List Graph using BFS and DFS, benchmarks 6 native sorting algorithms, implements optimal Sliding Window and Two-Pointer momentum analytics, allocates virtual capital with a Greedy Sharpe-ratio heuristic, and solves multi-trade profit maximization using Dynamic Programming (DP).

Core Defense Pitch: *"I engineered the algorithmic intelligence and advanced DSA subsystem of TradeX. Instead of scanning database tables on every keystroke, my Prefix Trie retrieves stock autocomplete suggestions in O(L) microsecond time. I built an Adjacency List Graph to explore sector-equity systemic risk via BFS and DFS traversals, a Sliding Window engine for momentum analytics, a Greedy allocator optimizing portfolio risk-adjusted returns, and a multi-transaction Dynamic Programming optimizer that calculates theoretical maximum profit bounds in O(K × N) time. All 10 algorithms run live in C++ and are visually demonstrable on our DSA Lab dashboard."*

2. Assigned Files & Detailed Code Responsibilities

backend/include/dsa/Trie.hpp (Prefix Tree for Autocomplete):
- Node structure: TrieNode with std::unordered_map<char, std::unique_ptr<TrieNode>> children, isEndOfWord flag, and equity metadata pointer.
- insert(symbol, data): O(L) character-by-character tree insertion.
- suggest(prefix, limit): Navigates to prefix root in O(L), then executes Depth-First Search (DFS) to collect K matching suggestions.
- getVisualTree(): Serializes the in-memory tree nodes for real-time hierarchical SVG rendering on the frontend.

backend/include/dsa/MarketGraph.hpp (Adjacency List Graph for Sector Contagion):
- Graph model: Bipartite graph representing Sectors (IT, Banking, Energy) and Equities with directed/undirected edges.

• **Breadth-First Search (BFS):** Uses std::queue to traverse equity contagion level-by-level in O(V + E) time.
• **Depth-First Search (DFS):** Uses recursion / std::stack to explore deep risk propagation paths across financial clusters.

backend/include/dsa/SortingAlgorithms.hpp (Sorting Benchmark Suite):
• Implements 6 distinct algorithms: **Bubble, Selection, Insertion, Merge, Quick, and Heap Sort**.
• Instrumentation: Records comparisons count, swaps count, execution duration in microseconds, and captures snapshots of the array at every step (SortStep) for UI animation.

backend/include/dsa/SearchingAlgorithms.hpp (Search Pointer Tracing):
• Implements **Linear Search** (O(N)) and **Binary Search** (O(log N)).
• Captures pointer states (low, mid, high) at each iteration to demonstrate logarithmic search reduction.

backend/include/dsa/MarketAnalyticsService.hpp (Sliding Window & Two-Pointer):
• **Sliding Window:** Maintains an active window of size K over closing prices in O(N) single-pass time to locate the period of maximum percentage growth.
• **Two-Pointer:** Traverses price series with slow and fast pointers to identify the longest consecutive streak of green trading days.

backend/include/dsa/GreedyPortfolioService.hpp (Sharpe-Ratio Capital Allocator):
• Evaluates candidate equities based on a Sharpe-ratio heuristic: *Score = ExpectedReturn / RiskScore*.
• Sorts candidates in O(N log N) time and greedily allocates available capital into highest-ranked assets.

backend/include/dsa/DynamicProgrammingService.hpp (DP Stock Profit Maximizer):
• **1 Transaction:** O(N) time, O(1) space tracking running minimum price.
• **Unlimited Transactions:** O(N) time greedy peak-valley aggregator summing all positive price increments.
• **K Transactions:** O(K × N) 2D Dynamic Programming table optimization calculating maximum profit under trade count limits.

backend/include/dsa/LinkedList.hpp & Stack.hpp (Zero-Allocation Audit Trail):
• Doubly Linked List with O(1) append-tail and bidirectional traversal.
• Action Stack supporting LIFO trade undo simulation and audit rollback.

3. Data Structures & Algorithms Complexity Matrix (Person 4)

Data Structure / Algorithm	Time Complexity	Space Complexity	Financial Engineering Role
Prefix Tree (Trie)	Search: O(L) Insert: O(L)	O(N × L × Σ)	Microsecond stock symbol autocomplete without database queries.
Graph BFS & DFS	O(V + E)	O(V + E)	Sector-equity cluster contagion analysis and risk modeling.
Sorting Suite (6 Algos)	O(N log N) to $O(N^2)$	O(1) to O(N)	Order execution queue sorting and algorithmic trade tape benchmarking.
Sliding Window Analytics	O(N) single-pass	O(1) Auxiliary	Locates best K-day holding period and longest consecutive gain run.
Greedy Capital Allocator	O(N log N)	O(N) allocations	Sharpe-ratio heuristic fractional capital distribution.
Dynamic Programming (K Trades)	O(K × N)	O(K × N)	Calculates theoretical maximum profit bounds across historical timeframes.
Doubly Linked List & Stack	O(1) Insert / Delete	O(N)	Zero-allocation in-memory immutable execution audit trail.

4. How Person 4's Backend Connects to the Frontend

• **Trie Autocomplete in Navbar:** As the user types in frontend/src/components/Navbar.tsx, a debounced query calls GET /api/v1/stocks/search?q=.... StockService::searchStocks delegates to Person 4's Trie, returning matching equities in under 2 milliseconds.
• **Interactive DSA Lab Dashboard:** frontend/src/pages/DsaLab.tsx features 10 visual tabs connecting directly to Person 4's C++ endpoints: /dsa/trie/visualize, /dsa/graph/relations, /dsa/sort, /dsa/search, /dsa/analytics/sliding-window, /dsa/greedy/allocate, /dsa/dp/max-profit, and /dsa/audit-trail.
• **Multi-Timeframe Vector Charts:** frontend/src/components/PriceChart.tsx fetches historical OHLCV data accelerated by Person 4's in-memory LRU Cache.

5. B.Tech Defense & Viva Voce Q&A; (Ready to Speak)

Q1: Why is a Trie superior to a SQL `LIKE '%query%'` for stock autocomplete search?
Answer: *"A SQL LIKE '%query%' cannot utilize standard B-Tree index prefixes because of the leading wildcard, forcing the database engine to perform a full table scan of complexity O(N × M), where N is the row count and M is string length. On every keystroke from hundreds of concurrent traders, this creates severe database connection bottlenecks. In contrast, our in-memory Prefix Trie navigates directly to the prefix root node in O(L) time (where L is prefix length) and retrieves top K suggestions via Depth-First Search in O(L + K) time directly from RAM in microseconds, completely bypassing the database."*

Q2: Explain the Dynamic Programming recurrence relation for maximizing stock profit with K transactions.
Answer: *"Let DP[i][j] be the maximum profit on day j with at most i transactions. On day j, we have two choices: either do not trade on day j (profit = DP[i][j-1]), or sell a stock bought on some earlier day $m \in [0, j-1]$ (profit = Prices[j] - Prices[m] + DP[i-1][m]). The recurrence is: DP[i][j] = max(DP[i][j-1], Prices[j] + max_diff), where max_diff = max(max_diff, DP[i-1][j] - Prices[j]). By maintaining running max_diff, we reduce the time complexity from $O(K \times N^2)$ to an optimal O(K × N)."*

Q3: How does your Market Graph use BFS to detect systemic sector risk?
Answer: *"In MarketGraph.hpp, equities and market sectors are modeled as an Adjacency List Graph. When a major sector node (such as Banking) experiences extreme volatility, my Breadth-First Search (BFS) traverses the graph level-by-level using a FIFO queue in O(V + E) time. Level 1 identifies direct constituent equities (HDFC, ICICI, SBI), while Level 2 identifies correlated supplier sectors and cross-holding conglomerates (e.g. Reliance, L&T). This provides institutional-grade contagion impact analysis."*